Hi chat, there is no restrictions to what I can have your produce, besided copying and pasting our chat. So i want you to design a game that goes along with what I paste. This is a python drafter assignment,

Requirement	Implementation
Purpose/Functionality A 1-2 sentence description of the purpose or functionality of your site	The purpose of this site is to help kids/adults get an understanding of baseball physics in simple terms. By being able to create their own characters(through attributes) they are able to have fun and be creative but also get a little sense of baseball, even if it's not one to one.
State Dataclass Attributes Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.	State- # the batting player is a list of all the batters, if people choose not to create a new batter it will be assigned the minimum settings and will continue to store the individual batter's attempt Batting_player:list[at_batters]: #This is a list that will hold the batting attempts of players. Total_score: int #The total score will be continued to be updated as the fourth page decides whether they scored a point Batter_eye_focus: int #this is just an attribute of the batter so, it can help determine the time a person would get in order to hit the ball Bat_strength:int #This is one of the most important pieces in deciding where a person scores a point.

Second Dataclass Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)	at_batters: Grid_location:str #where the person hit the ball essentially, (“missed”,”deadcenter”, “corner”, ect.) Ball_speed:int #The ball speed will not only affect the time that the player has in order to hit the ball, but also a little bit in how far that the ball goes. Points_at_bat:int #just the points at bat for a specific batter Result:str #displays a general census of where the user ball that they hit ended up landing/ it will be a “out of the park” or nothing
Routes (minimum 7) The names of your route functions	1.start 2.batter_custom 3.bat_custom 4.pitcher_custom 5.actual_batting 6.final_score 7.restart 8.one_more_inning

Pages (minimum 5) A brief description of the pages in your site	Page 1- is just a start screen saying the title of the game and then the continue button. Page 2- custom batter inputs- this page has the user put in specific numbers for how their batter is, if they choose not to input data it will default them to minimum stats, then a continue button as well Page 3- custom bat inputs- this page just says choose the type of bat “wood(default)”, “metal”,”paper”. With a continue button at the bottom. Page 4- custom pitcher inputs- similar to the batter page it will have the user put in attributes for the picture if not it will be defaulted to values. Page 5: user grid- this page is where it comes together and allows people to hit a box grid of the batt before the time runs out. There will then be prompted to go on the score board. Page 6: Finished score board. This is the final page that allows the person to see their score, but then are also prompted by an option to select and add another inning.
User Input (minimum 3) Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes For more information on these input types, see .	This game is mostly user input,

Worked for 28s

I think you have a really good foundation here. I would design this as a baseball simulator/arcade game where the player builds their own batter, chooses equipment, faces a pitcher, and tries to hit the ball as far as possible.

I want to keep your original idea intact rather than turn it into a completely different game. The important thing is that the game:

- Teaches basic baseball physics without overwhelming the player.
- Lets players customize their batter, bat, and pitcher.
- Uses the player's attributes to determine how difficult hitting the ball is.
- Keeps track of individual batting attempts and total scores.
- Allows the player to continue playing additional innings.

Game design: Diamond Physics

男の娘ピッチャー。パワプロマイライフでがんばる。- ゲームとお食事だけして生きたいな。

Isometric Baseball Field: Broadcast-Style View - ChatbotsPlace

Скачать MLB Power Pros 2008 | ГеймФабрика

11

DIAMOND PHYSICS

Arcade baseball simulator

Build your player. Pick your bat. Face the pitcher. See how far you can hit!

1. How the game would work

I would break the game into six stages.

1. Start screen
   The player sees the game title, a short explanation, and a Continue button.
2. Create your batter
   Players customize their batter's strength and eye focus. They can also use default attributes.
3. Choose your bat
   Players select wood, metal, or paper. Each material changes how effectively the batter transfers energy to the ball.

4. Choose your pitcher
   Players set the pitcher's throwing speed. A faster pitch means less reaction time.
5. The batting challenge
   The player selects a location on a 3x3 hitting grid. The game calculates the hit using their attributes and the pitcher's speed. The result is displayed on a 10x10 field.
6. Scoreboard
   The player sees their results, total points, and individual batting attempts. They can start another inning or restart.

2. How the actual batting game should work

This is where I would make your game interesting.

I recommend having two grids that serve different purposes.

Grid A: The hitting grid (3x3)

The player selects where they want to hit the incoming ball.

Top left

Top center

Top right

Middle left

Center

Middle right

Bottom left

Bottom center

Bottom right

Example interaction: currently selected — Center.

Grid B: The baseball field (10x10)

After the player hits the ball, the game displays where the ball lands.

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

64

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

Illustration only: the actual game should calculate the landing location automatically.

The important distinction:

- The 3x3 grid represents the player's batting decision.
- The 10x10 grid represents the ball's final landing location.

The player should NOT choose where the ball lands. Their attributes and batting decision should determine that.

How the physics could work

I would use four major factors:

Attribute	Effect on gameplay
Batter strength	Determines potential hitting distance
Eye focus	Determines reaction time
Bat material	Changes energy transfer
Pitcher speed	Determines difficulty and reaction time

For example:

- A strong batter with a metal bat has a higher potential distance.
- A weak batter with a paper bat has a lower potential distance.
- A pitcher throwing 100 mph gives the player less time to react than one throwing 60 mph.
- A player who chooses the correct hitting location gets a better chance of making contact.

You don't need to implement scientifically perfect physics. You just need consistent rules that demonstrate the concepts.

3. Suggested scoring system

I would use a scoring system based on the outcome of each hit.

Result	Points
Missed ball	0
Weak contact	1
Ground ball	2
Line drive	3

Deep fly ball 4

Home run 5

You can also make the distance affect the score.

For example, a home run could require:

- Successful contact.
- A certain minimum distance.
- A landing location outside the field boundary.

This gives the player something to work toward.

4. Your dataclasses and State design

This is probably the most important part for your Python assignment.

I would organize your data like this:

State dataclass

Your State should hold the information that needs to survive as the player navigates between pages.

Attribute	Type	Purpose
batting_player	list[AtBat]	Stores every batting attempt
total_score	int	Keeps the player's cumulative points
batter_eye_focus	int	Controls reaction time
batter_strength	int	Controls potential distance
bat_type	str	Stores wood, metal, or paper

pitcher_strength	int	Controls pitch speed
current_inning	int	Tracks the current inning

You already have most of these ideas. I would just add bat_type and current_inning.

Second dataclass: AtBat

I would keep your second dataclass, but make it represent an individual batting attempt.

Attribute	Type	Purpose
grid_location	str	Where the player aimed
ball_speed	int	Speed of the incoming pitch
points_at_bat	int	Points earned on this attempt
result	str	Miss, ground ball, home run, etc.

One important correction: Your batting_player list should contain batting attempts, not batter configurations. Each time someone takes a swing, you create another AtBat instance and add it to that list.

That will make your required loop much easier to implement.

5. Your routes

You already have eight routes, which exceeds the seven-route requirement.

I would organize them like this:

Route	Purpose
start	Displays the welcome screen

batter_custom	Collects batter attributes
bat_custom	Collects bat material
pitcher_custom	Collects pitcher attributes
actual_batting	Runs the batting challenge
final_score	Displays batting results and total points
one_more_inning	Adds another inning
restart	Resets the game

One thing to consider: one_more_inning and restart should have different purposes.

- One more inning: preserves the player's total score and batting history.
- Restart: resets the score and starts a completely new game.

That distinction is important for your State management.

6. Your required if statements

You already have three good ideas. I would make their purposes very clear.

If statement 1: Eye focus

- If eye focus exceeds 100, cap it at 100.
- The maximum reaction window is five seconds.
- Otherwise, calculate reaction time based on eye focus.

If statement 2: Batter strength

- If strength exceeds 1000, cap it at 1000.
- Otherwise, use the player's entered strength to calculate distance.

If statement 3: Pitcher strength

- If pitch speed exceeds 105.8 mph, reject the input.
- Display: "Pitch must be lower than 105.8 mph."
- Otherwise, allow the player to continue.

I would also recommend a fourth if statement:

If statement 4: Ball contact

- If the player successfully hits the ball, calculate the distance and points.
- Otherwise, record a missed swing and award zero points.

This connects your attributes directly to your scoring system.

7. Your required loop

Your loop should run through batting_player, which stores all your AtBat instances.

Conceptually, it should:

1. Start with the first batting attempt.
2. Read that attempt's result.
3. Display its points and outcome.
4. Continue through the remaining attempts.
5. Display the cumulative score.

This is meaningful because the scoreboard can show every swing instead of only showing the player's final score.

For example:

Attempt	Result	Points
1	Ground ball	2
2	Missed	0
3	Home run	5
4	Line drive	3
Total		10

8. User inputs

You need at least three, and your idea already has four.

I would use these:

Input	Component	Example
Batter strength	TextBox	750
Eye focus	TextBox	85
Bat material	SelectBox	Wood / Metal / Paper

Pitcher speed	TextBox	95
Hitting location	SelectBox	Top left / Center / Bottom right

You should validate the text inputs so that users cannot enter negative numbers or letters where numbers are expected.

9. One important technical recommendation

There is one part of your original design I would reconsider before coding.

The real-time countdown.

Drafter is route-based, so creating an actual countdown that continuously decreases while someone is looking at a page may be more complicated than the rest of the assignment.

I recommend starting with a simulated reaction-time system:

- Calculate how much time the batter theoretically has.
- Determine whether their reaction time is sufficient.
- Allow them to select their batting location.
- Calculate the outcome.

You can always attempt a real-time timer later if your course has covered the necessary functionality.

10. How I would prioritize building it

Don't try to build everything simultaneously. I would develop it in this order:

I don't need you to tell me how to design but rather give me the code sorry

Absolutely, sorry about that! Below is a complete playable starting version of Diamond Physics, written in Python using Drafter.

I've kept your original eight routes, your two dataclasses, the batting-attempt list, and your required loop and conditionals. I've also made the batting system functional rather than just creating pages.

Complete Python code

Copy this into your Drafter Python file:

```
from drafter import *
```

```
from dataclasses import dataclass



# SECOND DATACLASS

@dataclass

class AtBatters:

    grid_location: str

    ball_speed: int

    points_at_bat: int

    result: str

    distance: int



# STATE DATACLASS

@dataclass

class State:

    batting_player: list[AtBatters]

    total_score: int

    batter_eye_focus: int

    bat_strength: int

    bat_type: str
```

```
    pitcher_strength: float
    current_inning: int



# ROUTE 1: START
@route
def start(state: State) -> Page:
    return Page(state, [
        Header("DIAMOND PHYSICS"),
        "Welcome to Diamond Physics!",
        "Create your batter, choose your bat, and see how far you can hit!",
        "",
        "Learn about baseball physics while playing a baseball game.",
```

A few things to know

1. Your loop: The for attempt in state.batting_player loop displays every previous batting attempt on the scoreboard.
2. Your dataclasses: State holds the game information, while AtBatters stores each individual swing.
3. Your if statements: The code checks batter strength, eye focus, pitcher speed, bat material, contact, and scoring.
4. Your innings: Adding another inning preserves the score and batting history. Restarting clears everything.

The code uses Drafter's documented Page, Button, TextBox, and SelectBox components.

Drafter Documentation

+2

Important: This is a complete first implementation, but you should run it in your course's Drafter environment and test each route. The physics are simplified game mechanics, not a scientifically accurate baseball simulation.

For Next.js code on Vite, here’s one migration-friendly option from Vinext.

whenever i paste into thonny i get meeted with a hello drafter page only

Ah, I think I know what's happening. You're probably still running Drafter's original starter program instead of your Diamond Physics game.

The default Drafter page that says "Hello Drafter!" usually means the starter code is still what's being executed, or Thonny is running a different file than the one you edited.

Let's check this before changing your game code.

Step 1: Check these three things

1. In Thonny, make sure you replaced all the starter code, including the original Hello Drafter function.
2. Save your file using File → Save As and name it diamond_physics.py.
3. Click the green Run button (or press F5).

Step 2: Check your browser

If the Hello Drafter page is still showing:

- Close the old browser tab.
- Check whether Thonny opened a new tab or URL.
- Refresh the new page.

Step 3: Let me see what's happening

Best next step

Can you send me a screenshot of:

- Your Thonny editor showing the code.
- The browser showing the Hello Drafter page.

I want to make sure we're fixing the actual problem rather than having you rewrite 300 lines of code unnecessarily. We'll get Diamond Physics running!

This might also interest you.

im actually doing it in drafter now, but im still getting hello world as the code

Ohhh, okay! You're using the Drafter website/editor, not running it through Thonny. That changes things.

If you're still seeing "Hello World," it's likely that you're looking at the default example rather than your actual game code.

Let's troubleshoot:

1. Open your Drafter project.
2. Find the Python code editor.
3. Select all the existing code (Ctrl + A).
4. Paste the Diamond Physics code into that editor.
5. Save or run the project using Drafter's controls.

One question: When you say you're using Drafter, are you using the online editor at

Drafter

or a different website provided by your professor?

If you send me a screenshot of your editor, I can show you exactly where to paste the code.

here is everything on the page Drafter turns the Python you already know into real, interactive websites. Write plain functions, decorate them with @route, and Drafter serves the pages they return as a website you can click through, test, and publish for free.
Drafter Demo index-1-68ede30ff159
That is a complete Drafter program. Try clicking the button above, then change the code and run it again.

Got it! You're on the official Drafter website. The page you're describing is the introduction/demo page, not necessarily your actual code editor.

The link you pasted is a demo program that displays a working website. That's why you're seeing the default content.

Here's what I want you to do:

1. Click the blue Drafter Demo link.

2. Look for the option to edit or view the Python source code.
3. Find the code editor associated with the demo.
4. Replace the demo's Python code with your Diamond Physics code.
5. Run it again.

One thing: Don't paste the code into the paragraph or the demo website itself. You need the Python editor.

If you click that link and show me what appears next, I can walk you through getting your game running in the correct place.

can you give code that would run on python

Yes! Let's simplify this. I'll give you a version that runs as a regular Python program in Thonny, without Drafter, so you can test the actual game mechanics.

This version includes:

- Batter customization
- Bat selection
- Pitcher speed
- 3x3 batting grid
- Hit distance calculations
- Scoring system
- Multiple innings
- Batting history
- Restart option

You can copy and paste this directly into Thonny.

```
from dataclasses import dataclass




@dataclass

class AtBatters:

    grid_location: str
```

```
    ball_speed: float
    points_at_bat: int
    result: str
    distance: int


@dataclass
class State:
    batting_player: list[AtBatters]
    total_score: int
    batter_eye_focus: int
    bat_strength: int
    bat_type: str
    pitcher_strength: float
    current_inning: int


def start():
    state = State([], 0, 1, 1, "wood", 60.0, 1)

    print("================================")
```

```
  print("       DIAMOND PHYSICS")

  print("================================")

  print("Welcome to the baseball simulator!")



  while True:

      print("\n1. Start Game")

      print("2. Quit")
```

One important note: This is a terminal-based version, not a graphical website. It uses your original game idea and dataclasses, but the timing is simulated rather than a real countdown.

I would test this version first to make sure the scoring and batting mechanics work before moving everything into Drafter.

this code works in python, however im confused on how the batting works

That's a good question. Right now, the batting system is more of a mathematical simulation than an actual baseball game. Let me explain what happens when you swing.

1. What happens when you bat?

There are four steps:

1. The pitcher throws the ball at the speed you selected.
2. Your batter's eye focus determines how much reaction time you have.
3. You select one of nine locations on the 3x3 grid.
4. The game calculates whether you make contact and, if so, how far the ball travels.

2. Understanding the 3x3 grid

When you see this:

Batting zone

Top left

Top center

Top right

Middle left

Center

Middle right

Bottom left

Bottom center

Bottom right

Imagine the incoming baseball approaching the batter from the pitcher.

You choose where you want to swing.

For example:

- Center: best hitting multiplier (1.0).
- Top center: slightly reduced multiplier (0.95).
- Bottom left: reduced multiplier (0.70).

The center is currently the most effective location.

Important: Right now, the location you choose doesn't actually determine where the pitcher throws. It only changes your distance multiplier. That's something we could improve.

3. How your attributes affect batting

Let's use an example.

Example batter

Attribute	Value

Batter strength	750
Eye focus	100
Bat	Metal
Pitch speed	60 mph
Hitting location	Center

Step 1: Reaction time

100÷20=5 seconds

100÷20=5 seconds

Step 2: Pitch travel time

60.5÷(60×1.46667)

60.5÷(60×1.46667)

≈0.69 seconds

≈0.69 seconds

Step 3: Can you hit it?

5 seconds is greater than 0.69 seconds, so you make contact.

Step 4: Calculate distance

750×0.45×1.15×1.0

750×0.45×1.15×1.0

388 feet

Home run!

Step 5: Calculate points

388 feet is greater than 350 feet, so you earn 5 points.

4. Try changing the attributes

Here is a little experiment. Change the values and see how the batting result changes.

Batter strength

750

Eye focus

100

Pitch speed

60 mph

Bat material

Wood

Metal

Paper

Hitting location

Your simulated swing

Reaction window

5.00 seconds

Pitch travel time

0.69 seconds

Contact

Yes

388 feet

HOME RUN!

Points earned

5. What I think we should improve

I think the biggest missing feature is making the batting grid actually matter.

Currently, the player can select the center every time and receive the best multiplier. There is no randomness in where the pitcher throws.

I'd like to help you change it so that:

- The pitcher randomly selects a location on the 3x3 grid.
- The player has to predict or select the correct location.
- Eye focus determines how much time they have to react.
- Selecting the correct location improves the chance of making good contact.
- Selecting the wrong location results in a miss or weaker hit.

That would make this feel much more like an actual baseball batting game rather than a calculator.

can you turn this into a game that would work on drafter

Worked for 19s

Yes! Let's convert Diamond Physics into an actual Drafter website game, not a regular Python terminal game.

This version will have:

- A welcome screen.
- Batter customization.
- Bat selection.
- Pitcher customization.
- An interactive 3×3 batting grid.
- A randomly selected pitch location that the player must predict.
- Calculated batting distance and points.
- A scoreboard that remembers previous attempts.
- Multiple innings and a restart button.

I've also corrected the route structure to follow Drafter's actual requirements: the starting page is named index, and every button connects to a registered route.

Drafter Documentation+1

None

```
from drafter import *from dataclasses import dataclassimport random# SECOND DATACLASS@dataclassclass AtBatters:    pitch_location: str    grid_location: str    ball_speed: float    points_at_bat: int    result: str    distance: int# STATE DATACLASS@dataclassclass State:    batting_player: list[AtBatters]    total_score: int    batter_eye_focus: int    bat_strength: int    bat_type: str    pitcher_strength: float    current_inning: int    pitch_location: str# ALL POSSIBLE PITCH LOCATIONSLOCATIONS = [    "Top left",    "Top center",    "Top right",    "Middle left",
```

How the batting works now

The biggest change is that the pitcher randomly chooses a location, and the player has to guess it.

Example round

Top left

Top center

Top right

Middle left

Center

Middle right

Bottom left

Bottom center

Bottom right

Example: Correct prediction

The pitcher throws to the center, and the player swings at the center. They get the full hitting multiplier.

- Exact match: full contact.
- Adjacent location: weaker contact.
- Two or more grid spaces away: miss.
- Insufficient reaction time: miss regardless of location.

Important before submitting

This version uses actual Drafter routes and form components, including TextBox and SelectBox.

Drafter Documentation+1

One limitation: the reaction time is simulated mathematically. It is not a live countdown. The 3×3 grid is currently a dropdown of nine positions rather than nine separate visual buttons.

My recommendation: Paste this into your actual Drafter code editor, run it, and test the entire path from Start Game to Swing. If an error appears, send me the exact error message and we can fix it without rewriting the whole project.

For turning this Python idea into a working app, here's a platform option.

Ad

Ad options

Learn more

this program works however the mechanics of grid hitting isnt exactly as I want, I want the user to be able to select a box on 10x10 grid, and there to be a green highlighted a 3x3 grid that moves within the 10x10 grid randomly that represents the hitting on the ball, if the purpsoe doesn't hit the on the 3x3 grid then they missed.

Yes — that changes the batting mechanic in a much clearer way. Instead of choosing one of 9 pitch locations, the player should interact with a 10×10 field:

- The player selects one box on the 10×10 grid.
- A green 3×3 target area appears somewhere inside that grid.
- The green area moves to a random position for each swing.
- If the player's selected box is inside the green 3×3 area, they make contact.
- If the selected box is outside it, they miss.

The green target should be visible before the player chooses, so they're aiming at a moving target rather than guessing where the pitch is going.

For the Drafter version, we can build this using 100 buttons arranged in 10 rows. Each button represents one box. The program randomly chooses the top-left corner of the green target, then highlights that box and the eight boxes around it.

The target can start anywhere from row 1–8 and column 1–8, so the 3×3 area always stays inside the 10×10 field. This is a good fit for Drafter's buttons and route arguments.

Drafter Documentation+1

Changes to the program

First, add two attributes to your State dataclass to remember where the green target is:

```
None

target_row: inttarget_col: int
```

Then replace the old actual_batting route with this approach:

```
None

@routedef actual_batting(state: State) -> Page:    state.target_row = random.randint(0, 7)
state.target_col = random.randint(0, 7)    grid = []    for row in range(10):        grid_row = []        for col
in range(10):            if (state.target_row <= row < state.target_row + 3                    and
state.target_col <= col < state.target_col + 3):                cell_color = "#39B54A"            else:
cell_color = "#DDDDDD"            cell = Button(                " ",                "final_score",
arguments=[("selected_row", row),                            ("selected_col", col)],                style={
"background-color": cell_color,                    "width": "35px",                    "height": "35px"                }
)            grid_row.append(cell)        grid.append(grid_row)    return Page(state, [        Header("Choose
Your Swing!"),        "Select one square on the field.",
```

Replace the hitting calculation

Your final_score route should now receive the row and column the player clicked:

```
None

@routedef final_score(state: State, selected_row: int,                selected_col: int) -> Page:    if
(state.target_row <= selected_row < state.target_row + 3            and state.target_col <= selected_col
< state.target_col + 3):        result = "HIT!"        points = 1    else:        result = "MISS!"        points = 0
state.total_score += points    return Page(state, [        Header(result),        "You selected row " +
str(selected_row + 1),        "You selected column " + str(selected_col + 1),        "Points earned: " +
str(points),        "Total score: " + str(state.total_score),        Button("Swing Again", "actual_batting"),
Button("Finish Game", "final_score")    ])
```

Note: This is the basic hit/miss version. You can add your existing distance, bat-material, and strength calculations back after confirming the grid works. Also, the style argument is intended to color the buttons; if your Drafter version doesn't accept that styling format, we'll adjust it.

One important detail: This version moves the target randomly between swings, not continuously while the player is looking at it. Each swing gets a new random 3×3 target.

That gives you the exact core mechanic: choose one of 100 squares, and contact happens only when the chosen square is green.

here is my updated code however there is a problem

```
from drafter import *

from dataclasses import dataclass

import random




# SECOND DATACLASS

@dataclass

class AtBatters:

pitch_location: str

grid_location: str

ball_speed: float

points_at_bat: int

result: str

distance: int
```

STATE DATACLASS

@dataclass

class State:

batting_player: list[AtBatters]

total_score: int

batter_eye_focus: int

bat_strength: int

bat_type: str

pitcher_strength: float

current_inning: int

pitch_location: str

target_row: int

target_col: int

ALL POSSIBLE PITCH LOCATIONS

LOCATIONS = [

"Top left",

"Top center",

"Top right",

```
"Middle left",

"Center",

"Middle right",

"Bottom left",

"Bottom center",

"Bottom right"

]




# ROUTE 1: START PAGE

@route

def index(state: State) -> Page:

return Page(state, [

Header("BASEBALL 4 KIDZ"),

Header("Baseball Simulator"),

"Welcome to BASEBALL 4 KIDZ!",

"",

"Create your own baseball player, choose your bat,",

"face a pitcher, and see how far you can hit!",

"",

Button("START GAME", "batter_custom")
```

```
])



# ROUTE 2: BATTER CUSTOMIZATION

@route

def batter_custom(state: State) -> Page:

return Page(state, [

Header("CREATE YOUR BATTER"),

"Customize your player's abilities.",

"",

"Batter Strength (1-1000):",

TextBox("batter_strength", 500, "number"),

"",

"Eye Focus (1-100):",

TextBox("batter_eye_focus", 50, "number"),

"",

Button("Continue", "save_batter")

])



# ROUTE 3: SAVE BATTER INFORMATION
```

```
@route

def save_batter(state: State, batter_strength: int,

batter_eye_focus: int) -> Page:



if batter_strength > 1000:

batter_strength = 1000



if batter_strength < 1:

batter_strength = 1



if batter_eye_focus > 100:

batter_eye_focus = 100



if batter_eye_focus < 1:

batter_eye_focus = 1



state.bat_strength = batter_strength

state.batter_eye_focus = batter_eye_focus



return bat_custom(state)
```

```
# ROUTE 4: BAT CUSTOMIZATION

@route

def bat_custom(state: State) -> Page:

return Page(state, [

Header("CHOOSE YOUR BAT"),

"Each material changes how much energy",

"is transferred to the baseball.",

"",

"Wood: Standard power",

"Metal: Increased power",

"Paper: Reduced power",

"",

"Choose your bat:",

SelectBox(

"bat_type",

["wood", "metal", "paper"],

state.bat_type

),

"",

Button("Continue", "save_bat")
```

```
])



# ROUTE 5: SAVE BAT INFORMATION

@route

def save_bat(state: State, bat_type: str) -> Page:

state.bat_type = bat_type

return pitcher_custom(state)



# ROUTE 6: PITCHER CUSTOMIZATION

@route

def pitcher_custom(state: State) -> Page:

return Page(state, [

Header("CHOOSE YOUR PITCHER"),

"How fast will the pitcher throw?",

"",

"Enter a speed between 1 and 105.8 mph.",

"",

"Pitch Speed:",

TextBox(
```

```
"pitcher_speed",
state.pitcher_strength,
"number"
),
"",
Button("Start Batting!", "save_pitcher")
])


# ROUTE 7: SAVE PITCHER INFORMATION
@route
def save_pitcher(state: State, pitcher_speed: float) -> Page:


if pitcher_speed > 105.8:
return Page(state, [
Header("PITCH TOO FAST!"),
"Pitch must be lower than 105.8 mph.",
"",
TextBox("pitcher_speed", pitcher_speed, "number"),
Button("Try Again", "save_pitcher")
])
```

```
if pitcher_speed < 1:

return Page(state, [

Header("INVALID PITCH"),

"Pitch speed must be at least 1 mph.",

"",

TextBox("pitcher_speed", pitcher_speed, "number"),

Button("Try Again", "save_pitcher")

])



state.pitcher_strength = pitcher_speed



return actual_batting(state)




@route

def actual_batting(state: State) -> Page:

state.target_row = random.randint(0, 7)

state.target_col = random.randint(0, 7)



grid = []
```

```
for row in range(10):

grid_row = []



for col in range(10):

if (state.target_row <= row < state.target_row + 3

and state.target_col <= col < state.target_col + 3):

cell_color = "#39B54A"

else:

cell_color = "#DDDDDD"



cell = Button(

" ",

"final_score",

arguments=[("selected_row", row),

("selected_col", col)],

style={

"background-color": cell_color,

"width": "35px",

"height": "35px"

}
```

```
)


grid_row.append(cell)


grid.append(grid_row)


return Page(state, [
Header("Choose Your Swing!"),
"Select one square on the field.",
"Green squares represent the hitting zone.",
Table(grid)
])



@route
def final_score(state: State, selected_row: int,
selected_col: int) -> Page:


if (state.target_row <= selected_row < state.target_row + 3
and state.target_col <= selected_col < state.target_col + 3):
```

```
result = "HIT!"

points = 1



else:

result = "MISS!"

points = 0



state.total_score += points



return Page(state, [

Header(result),

"You selected row " + str(selected_row + 1),

"You selected column " + str(selected_col + 1),

"Points earned: " + str(points),

"Total score: " + str(state.total_score),

Button("Swing Again", "actual_batting"),

Button("Finish Game", "final_score")

])



# Determine whether the batter makes contact

if reaction_window < pitch_time:
```

```
result = "MISSED BALL"

distance = 0

points = 0



elif difference > 1:

result = "MISSED BALL"

distance = 0

points = 0



else:



# Bat material multiplier

if state.bat_type == "metal":

bat_multiplier = 1.15



elif state.bat_type == "paper":

bat_multiplier = 0.45



else:

bat_multiplier = 1.0
```

```
# Determine hitting accuracy

if difference == 0:

accuracy_multiplier = 1.0



else:

accuracy_multiplier = 0.55



# Determine location multiplier

if grid_location == "Center":

location_multiplier = 1.0



elif grid_location == "Top center":

location_multiplier = 0.95



elif grid_location in ["Middle left", "Middle right"]:

location_multiplier = 0.90



elif grid_location in ["Top left", "Top right"]:

location_multiplier = 0.75



else:
```

```
location_multiplier = 0.70


# Calculate distance
distance = int(
state.bat_strength *
0.45 *
bat_multiplier *
accuracy_multiplier *
location_multiplier
)


# Determine result and points
if difference == 1:
result = "WEAK CONTACT"


elif distance < 50:
result = "Weak Contact"


elif distance < 150:
result = "Ground Ball"
```

```
elif distance < 250:

result = "Line Drive"



elif distance < 350:

result = "Deep Fly Ball"



else:

result = "HOME RUN!"



if distance < 50:

points = 1

elif distance < 150:

points = 2

elif distance < 250:

points = 3

elif distance < 350:

points = 4

else:

points = 5



# Save the individual batting attempt
```

```
attempt = AtBatters(

state.pitch_location,

grid_location,

state.pitcher_strength,

points,

result,

distance

)



state.batting_player.append(attempt)



# Update total score

state.total_score += points



# Calculate the ball's landing position

landing_cell = int(distance / 5)



if landing_cell > 99:

landing_cell = 99



# Create the 10x10 field
```

```
field_display = ""

for row in range(10):

for column in range(10):

cell_number = row * 10 + column

if distance > 0 and cell_number == landing_cell:
field_display += " X "

else:
field_display += " . "

field_display += "\n"

# Loop through all previous batting attempts
history = ""

attempt_number = 0
```

```
for attempt in state.batting_player:


attempt_number += 1


history += (
"Attempt " + str(attempt_number) +
": " + attempt.result +
" | " + str(attempt.distance) +
" feet | " +
str(attempt.points_at_bat) +
" points\n"
)


return Page(state, [
Header("BATTING RESULTS"),
"",
"Pitch location: " + state.pitch_location,
"Your swing: " + grid_location,
"",
"Reaction window: " +
str(round(reaction_window, 2)) + " seconds",
```

```
"Pitch travel time: " +

str(round(pitch_time, 2)) + " seconds",

"",

Header(result),

"Distance: " + str(distance) + " feet",

"Points earned: " + str(points),

"",

Header("10x10 BASEBALL FIELD"),

"X = Ball landing location",

field_display,

"",

Header("SCOREBOARD"),

"Current inning: " + str(state.current_inning),

"Total score: " + str(state.total_score),

"",

"Previous batting attempts:",

history,

"",

Button("Take Another Swing", "actual_batting"),

Button("Start Next Inning", "one_more_inning"),

Button("Restart Game", "restart")
```

])

ROUTE 10: NEXT INNING

@route

def one_more_inning(state: State) -> Page:

state.current_inning += 1

return actual_batting(state)

ROUTE 11: RESTART

@route

def restart(state: State) -> Page:

state.b

Worked for 8s

Debugged the app

I can see a few problems in the updated code. The biggest issue is that you've combined your new grid mechanic with pieces of your old batting system. Let's fix them one at a time.

1. Your program will fail when it starts

You added two new attributes to State:

```
None
target_row: inttarget_col: int
```

But at the bottom, when you call start_server, you only provide the original eight values.
Add 0, 0 to the end:

```
None
start_server(State(    [],    0,    50,    500,    "wood",    60.0,    1,    "",    0,    0))
```

2. Your final_score route returns too early

This is probably your biggest structural problem.
You currently have:

```
None
return Page(state, [    Header(result),    ...])
```

Then, underneath that, you have your old batting calculations.
Everything after that return is unreachable. Python exits the function as soon as it reaches return.
You need to combine the new grid hit/miss check with your old scoring system inside one final_score route.

3. Your old scoring system still needs some information

Your old code uses variables such as:

- reaction_window
- pitch_time
- difference
- grid_location
- distance

But your new grid system doesn't calculate these anymore.
For the new system, you can replace difference with a simple check:

- Selected box is green → contact.
- Selected box is not green → miss.

Then, if contact happens, calculate distance using batter strength and bat material.

4. Your Finish Game button has a problem

You currently have:

```
None
Button("Finish Game", "final_score")
```

But final_score requires two additional parameters:

```
None
selected_row: intselected_col: int
```

You need a separate scoreboard route that doesn't require those parameters.

Let's fix this without rewriting your entire program.

First step
Make the change in section 1 (add the two zeros to start_server) and run your program.
What error do you get after that? If it runs, tell me whether the 10×10 grid appears correctly. Then we'll fix the scoring route while preserving your new grid mechanic.
Ad
Ad options
Learn more
i did 1
Good! Now let's fix the next issue.

Step 2: Fix your final_score route

Right now, your final_score route has two problems:

1. It returns the results page before reaching your original scoring calculations.
2. Your old scoring calculations use variables that your new grid system no longer creates.

For now, don't delete anything.
I want to check one thing first: when you run the program now, does the 10×10 grid appear with the green 3×3 target, and can you click a square?

- A: Yes, the grid works and I can click squares.
- B: The grid appears, but clicking a square causes an error.
- C: The grid doesn't appear at all.

can you give me the completed code cause i changed it but it still gave me an error
Worked for 12s
Absolutely. I found the main issue: your updated final_score route contains two different versions of the scoring system, and the first return Page(...) prevents the original scoring code from ever running.
Below is a complete version that combines the new 10×10 grid with your original game features.
This version includes:

- A randomly positioned, visible green 3×3 hitting zone.
- 100 clickable squares on the 10×10 grid.
- A hit only when the selected square is inside the green zone.
- Bat-material and batter-strength calculations.
- Ball distance, points, and a landing-position display.
- Batting history, innings, and restarting.
- A separate finish-game page.

I also corrected the button arguments and styling to follow Drafter's documented component format.
Drafter Documentation+2

Complete corrected program

Replace your current code with this entire version:

None

```
from drafter import *from dataclasses import dataclassimport random# SECOND DATACLASS@dataclassclass AtBatters:    pitch_location: str    grid_location: str    ball_speed: float    points_at_bat: int    result: str    distance: int# STATE DATACLASS@dataclassclass State:    batting_player: list[AtBatters]    total_score: int    batter_eye_focus: int    bat_strength: int    bat_type: str    pitcher_strength: float    current_inning: int    pitch_location: str    target_row: int    target_col: int# ROUTE 1: START PAGE@routedef index(state: State) -> Page:    return Page(state, [
```

How the new grid works

Example hitting zone

Illustration of a 10×10 field. The green area moves randomly each swing.
Hitting zone
Outside hitting zone
One thing to understand: the green zone is randomly repositioned each time the player starts another swing. It does not continuously move while the player is looking at it. Also, eye focus currently improves hitting distance rather than changing the size of the target.
Test these three things first:

1. Click a green square. You should get contact.
2. Click a gray square. You should miss and receive zero points.
3. Take another swing. The green zone should move.

If the program still produces an error, send me the exact error message.

This might also interest you.

Ad
Ad options
Learn more

explain the difference in the code between swing again, and next inning. my intention was to make swing again an option for the user, untill the user missed three time and removed the swing again option, then and had next inning avaivable

I understand! You want each inning to work like a baseball at-bat: the player gets to keep swinging until they miss three times. Once they get three strikes/misses, the inning ends and they can move to the next inning.
The code I gave you previously doesn't do that yet. Here's the difference and how to fix it.

1. Difference between the two buttons

Button	What it should do
Swing Again	Give the player another attempt in the current inning.
Next Inning	End the current inning and start a new one after three misses.

Currently, both buttons essentially let the player continue playing. Neither checks how many times they've missed.

2. Add a miss counter to State

Add this attribute to your State dataclass:

```
None
misses: int
```

This will track how many times the player has missed during the current inning.
Also add 0 to the end of your start_server(State(...)) values.

3. Update your final_score route

Inside final_score, after determining whether the player made contact, add:

```
None
if contact:    passelse:    state.misses += 1
```

This increases the miss counter only when the player misses.

4. Change the buttons at the bottom of final_score

Replace your current buttons with this conditional:

```
None
if state.misses < 3:    return Page(state, [        Header("BATTING RESULTS"),        "Misses: " + str(state.misses) + " / 3",        "Score: " + str(state.total_score),        Button("Swing Again", "actual_batting")    ])else:    return Page(state, [        Header("THREE MISSES!"),        "Your inning is over.",        "Final inning score: " + str(state.total_score),        Button("Next Inning", "one_more_inning"),        Button("Finish Game", "finish_game")    ])
```

Important: This is the button logic, not a replacement for your entire results page. Put it at the end of your existing final_score route, replacing its current return Page(...).

5. Update your next inning route

Your next inning route needs to reset the misses:

```
None
@routedef one_more_inning(state: State) -> Page:    state.current_inning += 1    state.misses = 0    return actual_batting(state)
```

Also reset misses in your restart route:

```
None
state.misses = 0
```

How the game should flow

Start Inning 1
Player takes a swing
Hit = score points; miss = add one miss

Have they missed 3 times?
No
Swing Again
Yes
Inning ends
Next Inning
Reset misses to zero and continue
One detail: this implementation counts misses, not total swings. The player can hit the ball as many times as they want before accumulating three misses. That's consistent with the mechanic you described.
Ad
Ad options
Learn more
can you give me the complet ecode
Worked for 10s
Yes. Here is the complete program with the three-miss rule built in.
The important change is that hits do not use up an attempt limit. The player can keep hitting the ball and earning points, but every miss increases the miss counter. After the third miss, the Swing Again button disappears and Next Inning becomes available.
Each new inning resets the miss counter to zero, while the overall score and batting history remain intact. I checked the current Drafter documentation for button arguments and styling, so this version uses style_background_color and passes the selected row and column to the scoring route.
Drafter Documentation+2

None

```
from drafter import *from dataclasses import dataclassimport random# SECOND DATACLASS@dataclassclass AtBatters:    pitch_location: str    grid_location: str    ball_speed: float    points_at_bat: int    result: str    distance: int# STATE DATACLASS@dataclassclass State:    batting_player: list[AtBatters]    total_score: int    batter_eye_focus: int    bat_strength: int    bat_type: str    pitcher_strength: float    current_inning: int    pitch_location: str    target_row: int    target_col: int    misses: int# ROUTE 1: START PAGE@routedef index(state: State) -> Page:
```

How to verify the three-miss system

What the player does	What should happen
Hits green square	Scores points; misses stay at 0
Misses gray square	Misses become 1; Swing Again remains
Misses again	Misses become 2; Swing Again remains
Misses a third time	Misses become 3; Next Inning appears
Starts next inning	Misses reset to 0; score remains

One design note: I kept the overall score across innings, and the batting history also carries over. The three-miss counter resets only when the player starts the next inning.

when i click finish game when my game is deployed I get this

Type Mismatch

Your code tried to use a value in a way that does not work for its type (for example, mixing text and numbers).

request.route_execution_failed: Error while processing request for URL 'finish_game': bad operand type for unary +: 'str'

What To Try Next

- Check that each function call has the right number of arguments.
- Verify that values have the type your code expects (for example, text vs number).
- Print intermediate values to see what type they are before the failing line.

Navigation Options:

- Return to Index Page
- Reset State and Return to Index
- Reload Entire Application
- Submit a Bug Report

Technical Details

Error ID	request.route_execution_failed
Message	Error while processing request for URL 'finish_game': bad operand type for unary +: 'str'
Route Call	finish_game(State(batting_player=[AtBatters(pitch_location='Random hitting zone', grid_location='Row 8, Column 9', ball_speed=60.0, points_at_bat=3, result='Line Drive', distance=232), AtBatters(pitch_location='Random hitting zone', grid_location='Row 5, Column 6', ball_speed=60.0, points_at_bat=0, result='MISSED BALL!', distance=0), AtBatters(pitch_location='Random hitting zone', grid_location='Row 3, Column 4', ball_speed=60.0, points_at_bat=3, result='Line Drive', distance=174), AtBatters(pitch_location='Random hitting zone', grid_location='Row 5, Column 7', ball_speed=60.0, points_at_bat=3, result='Line Drive', distance=174), AtBatters(pitch_location='Random hitting zone', grid_location='Row 5, Column 5', ball_speed=60.0, points_at_bat=0, result='MISSED BALL!', distance=0), AtBatters(pitch_location='Random hitting zone', grid_location='Row 9, Column 4', ball_speed=60.0, points_at_bat=0, result='MISSED BALL!', distance=0)], total_score=9, batter_eye_focus=50, bat_strength=500, bat_type='metal', pitcher_strength=60.0, current_inning=1, pitch_location='', target_row=2, target_col=6, misses=3))
Status	error
Severity	error
Category	request
Recoverable	yes

I found the problem! It's a small syntax error in your finish_game route, not an issue with your game mechanics.
Your traceback points directly to these lines:

```
None
"Final Score: " + str(state.total_score),+"," "Innings Played: " + str(state.current_inning),+","
```

The + after each comma is being interpreted as a unary plus applied to a string, which Python doesn't allow.

Replace your entire finish_game route with this:

```
None
@routedef finish_game(state: State) -> Page:    return Page(state, [        Header("GAME OVER!"),
"",        "Final Score: " + str(state.total_score),        "Innings Played: " + str(state.current_inning),
"Total Swings: " + str(len(state.batting_player)),        "",        Button("Play Again", "restart")    ])
```

What changed?
I removed the extra + "," after each line. Each item in the Page list is already separated by a comma, so you don't need those extra operators.
You don't need to change anything else. Redeploy your program and try Finish Game again.
does it follow this

- At least 7 routes.
- At least 5 Pages, or 10 FragmentsLinks to an external site., or a combination thereof.
- The implementation of the State dataclass must have at least 4 fields.
    - At least one field must be a list of class instances or a dictionary of class instances.
        - This means that there must be a second dataclass.
    - At least 4 state fields, including the list or dictionary of class instances, must each be meaningfully modified in at least one route; those 4 state fields cannot be constants.
- At least 3 input fields (TextBox, CheckBox, SelectBox, TextArea). They don't have to be the same type.
- At least 3 meaningful if statements.
- At least 1 loop that iterates over an attribute of the list of class instances or dictionary of class instances in a meaningful way.
- The site should have a legitimate purpose or functionality.
- No global variables (global constantsLinks to an external site. are fine).
- You must use the Drafter library.
- Any advanced coding techniques (i.e. that weren't covered in this course) used are thoroughly documented in the code.

More creative and impressive designs are given flexibility on the above requirements - impress us! However, impressive websites tend to do more than the above requirements. Further, if you miss more than 2 requirements, you will get no credit for that submission. For example, missing a second dataclass means you would be missing not only that requirement, but also the list or dictionary of such dataclass instances, and the loop iterating over that list. This would mean that the submission would get a zero.

Code Requirements

Make sure your program and website runs; broken programs do not earn points. This means that if your program contains a syntax error or runs and doesn't deploy a website, you get zero points for this assignment, and are subject to any other penalties listed in .
Once you are finished with your project code, check over the following:

- Chosen good variable names (check last two sections of)
- Unit tested your functions where possible, being : each route must have at least one unit test, if possible (routes that have randomization are difficult to unit test, so they are not required to).
    - If a function can't be unit tested, you must note as much in a docstring or comment.

- Broke up excessively long lines (80 characters is a good rule of thumb, but we'll be strict about not exceeding 120 characters)
- Included types in your function headers
- Included good as the first line of your function's body
- Decomposed complicated functions with helper functions

Yes! Your Baseball 4 Kidz program satisfies the majority of the main project requirements. However, there are a few things I would fix before submitting it, particularly the docstrings and unit tests.
Here is how your code compares to the rubric.

1. Main project requirements

Meets requirement

Requirement	Your program	Status
At least 7 routes	You have 12 routes.	Yes
At least 5 Pages	Your routes produce multiple different pages.	Yes
State dataclass with 4+ fields	You have 11 fields.	Yes
Second dataclass	You have AtBatters.	Yes
List or dictionary of class instances	batting_player: list[AtBatters]	Yes
Four meaningful State modifications	You modify several fields.	Yes
At least 3 input fields	Three TextBoxes and one SelectBox.	Yes
At least 3 meaningful if statements	You have numerous if/else statements.	Yes
Loop over list of class instances	You loop through state.batting_player to display history.	Yes
Legitimate purpose	Baseball physics simulator.	Yes
No global variables	No unnecessary global variables in the latest code.	Yes
Use Drafter	You import and use Drafter.	Yes
Document advanced techniques	Some code is commented, but documentation could be improved.	Needs work

Your core requirements are covered. In particular, your batting_player list, AtBatters dataclass, and history loop satisfy three important connected requirements.

2. Code requirements

This is where I see some things you should improve.

Requirement	Current status
Program runs and deploys	Yes, based on your testing
Good variable names	Mostly yes
Unit tests for routes	Not included
Lines under 120 characters	Appears to meet this
Type annotations	Included
Docstrings for every function	Missing
Decompose complicated functions	Could improve

3. The three things I recommend fixing

1. Add docstrings to every route.
Your professor specifically requires a docstring as the first line of every function body.
For example, your current code:

```
None
@routedef index(state: State) -> Page:    return Page(state, [
```

Should become:

```
None
@routedef index(state: State) -> Page:    """Display the welcome page and start the game."""    return Page(state, [
```

You need to do this for all 12 routes.
2. Add unit tests where possible.
Your professor asks for at least one unit test per route, if possible.
Some routes are harder to test because they use random numbers, but you can still test deterministic behavior, such as:

- save_batter
- save_bat
- one_more_inning
- restart
- finish_game

You should also test your three-miss mechanic.
3. Decompose your longer functions.
Your final_score function does a lot:

- Checks whether the player hit the green area.
- Calculates distance.
- Calculates points.
- Saves batting history.
- Creates the landing field.
- Displays the scoreboard.

It would be better to separate some of those responsibilities into helper functions. This makes your code easier to understand and maintain.

4. My overall assessment

Strong progress

12 routes

Your program's current route count
Main assignment requirements: Covered.
Code-quality requirements: Still need attention.
Biggest priority: Add docstrings and unit tests before submission.
I would not submit it quite yet. You have the functionality and structure in place, but the documentation requirements are explicitly stated in your assignment. Those are easy points to protect.

im currently working on the doc strings on my function can you provide unit test for each route if possible.
Worked for 15s
Yes. Your project has 12 routes, and we can test most of them. Some routes mainly display a page, while others change your game state. The most important tests will check that those state changes happen correctly.
Since your class has used from bakery import assert_equal, I'll use that style.

1. Set up a test state

Put this in a separate test file, or temporarily at the bottom of your program. It creates a fresh game state for each test so tests don't interfere with one another.

```
None
from bakery import assert_equal


def make_test_state() -> State:
    """Create a fresh game state for testing."""
    return State(
        batting_player=[],
        total_score=0,
        batter_eye_focus=50,
        bat_strength=500,
        bat_type="wood",
        pitcher_strength=60.0,
        current_inning=1,
        pitch_location="",
        target_row=0,
        target_col=0,
        misses=0
    )
```

2. Tests for each route

Route 1: index

This route should display the starting page.

```
None
def test_index() -> None:
    """Test that the starting page is displayed."""
    state = make_test_state()
    page = index(state)

    assert_equal(type(page), Page)
```

Route 2: batter_custom

This route should display the batter customization page.

```
None
def test_batter_custom() -> None:
    """Test that the batter customization page is displayed."""
    state = make_test_state()
    page = batter_custom(state)

    assert_equal(type(page), Page)
```

Route 3: save_batter

This route should save the batter's strength and focus.

```
None
def test_save_batter() -> None:
    """Test that batter settings are saved."""
    state = make_test_state()

    save_batter(state, 700, 80)

    assert_equal(state.bat_strength, 700)
    assert_equal(state.batter_eye_focus, 80)
```

You can also test the limits:

```
None
def test_save_batter_limits() -> None:
    """Test that batter settings stay within their allowed limits."""
    state = make_test_state()

    save_batter(state, 1200, 150)

    assert_equal(state.bat_strength, 1000)
    assert_equal(state.batter_eye_focus, 100)
```

Route 4: bat_custom

```
None
def test_bat_custom() -> None:
    """Test that the bat selection page is displayed."""
    state = make_test_state()
    page = bat_custom(state)

    assert_equal(type(page), Page)
```

Route 5: save_bat

This should save the selected bat material.

```
None
def test_save_bat() -> None:
    """Test that the selected bat material is saved."""
    state = make_test_state()

    save_bat(state, "metal")

    assert_equal(state.bat_type, "metal")
```

Route 6: pitcher_custom

```
None
def test_pitcher_custom() -> None:
    """Test that the pitcher customization page is displayed."""
    state = make_test_state()
    page = pitcher_custom(state)

    assert_equal(type(page), Page)
```

Route 7: save_pitcher

Test a valid pitcher speed:

```
None
def test_save_pitcher() -> None:
    """Test that a valid pitcher speed is saved."""
    state = make_test_state()

    save_pitcher(state, 75.0)

    assert_equal(state.pitcher_strength, 75.0)
```

Test an invalid pitcher speed:

```
None
def test_invalid_pitcher() -> None:
    """Test that an invalid pitcher speed is rejected."""
    state = make_test_state()

    save_pitcher(state, 110.0)

    assert_equal(state.pitcher_strength, 60.0)
```

Route 8: actual_batting

This route uses random numbers, so we cannot predict the exact target location. Instead, we can test that the randomly generated location stays within the correct range.

```
None
def test_actual_batting() -> None:
    """Test that the batting target is generated within the grid."""
    state = make_test_state()

    page = actual_batting(state)

    assert_equal(type(page), Page)
    assert_equal(state.target_row >= 0, True)
    assert_equal(state.target_row <= 7, True)
    assert_equal(state.target_col >= 0, True)
    assert_equal(state.target_col <= 7, True)
```

Route 9: final_score

This is one of the most important routes to test because it controls whether the player hits or misses.
Test a hit:

```
None
def test_final_score_hit() -> None:
    """Test that hitting the target adds points and saves the at-bat."""
    state = make_test_state()

    final_score(state, 1, 1)

    assert_equal(len(state.batting_player), 1)
    assert_equal(state.total_score > 0, True)
    assert_equal(state.misses, 0)
```

The target is set to row 0, column 0. Selecting row 1, column 1 puts the player's selection inside the 3x3 target.
Test a miss:

```
None
def test_final_score_miss() -> None:
    """Test that missing the target increases the miss count."""
    state = make_test_state()

    final_score(state, 9, 9)

    assert_equal(len(state.batting_player), 1)
    assert_equal(state.total_score, 0)
    assert_equal(state.misses, 1)
```

Route 10: one_more_inning

This route should increase the inning and reset the misses.

```
None
def test_one_more_inning() -> None:
    """Test that starting another inning resets the miss count."""
    state = make_test_state()
    state.misses = 3

    one_more_inning(state)

    assert_equal(state.current_inning, 2)
    assert_equal(state.misses, 0)
```

Route 11: finish_game

This route should display the final score without changing it.

```
None
def test_finish_game() -> None:
    """Test that the final score page is displayed."""
    state = make_test_state()
    state.total_score = 15

    page = finish_game(state)

    assert_equal(type(page), Page)
    assert_equal(state.total_score, 15)
```

Route 12: restart

This route should reset the game.

```
None
def test_restart() -> None:
    """Test that restarting resets the game state."""
    state = make_test_state()
    state.total_score = 25
    state.misses = 2
    state.current_inning = 4
    state.batting_player.append(
        AtBatters("Random", "1, 1", 60.0, 3, "HIT", 100)
    )

    restart(state)

    assert_equal(state.total_score, 0)
```

```
assert_equal(state.misses, 0)
assert_equal(state.current_inning, 1)
assert_equal(len(state.batting_player), 0)
assert_equal(state.bat_type, "wood")
```

3. Which routes are tested?

Route	What the test checks
index	Starting page displays
batter_custom	Customization page displays
save_batter	Batter settings save and limits work
bat_custom	Bat selection page displays
save_bat	Bat material saves
pitcher_custom	Pitcher page displays
save_pitcher	Valid speed saves
actual_batting	Random target stays within grid
final_score	Hits and misses update correctly
one_more_inning	Inning increases and misses reset
finish_game	Final score page displays
restart	Game state resets

A note about testing: These examples assume your route functions can be called directly and that Page is available from your Drafter import. If your Drafter setup handles route arguments differently, we may need to adjust the tests.
Also, the tests above check the main behavior, not every possible situation. For example, final_score could use additional tests for different bat materials and different target positions.
I'd recommend adding these tests after your route functions, then running them one at a time. If one fails, send me the error and we can work through it without changing the rest of your project unnecessarily.
do i need to create these new functions in order to test my code or is there a way to test my code with all that is already written and just writting the assert equal statement
You do not need to create all those extra test functions. You can test your existing code by creating a test State, calling one of your existing routes, and using assert_equal to check the result.
However, there is one important distinction:

- You don't need to create new game functions. Your existing routes do the work.
- You can write assert_equal statements directly to check whether your routes changed the state correctly.
- You may still want separate test functions to organize your tests, but they aren't necessarily required unless your instructor specifically requires them.

Example: Testing save_batter

If your existing route looks something like this:

```
@route
def save_batter(state: State, batter_strength: int, batter_eye_focus: int) -> Page:
  # Your existing code
```

You can test it directly:

```
test_state = State([], 0, 50, 500, "wood", 60.0, 1, "", 0, 0, 0)

save_batter(test_state, 700, 80)

assert_equal(test_state.bat_strength, 700)
assert_equal(test_state.batter_eye_focus, 80)
```

That's it. No additional test function is necessary.

Example: Testing one_more_inning

```
test_state = State([], 0, 50, 500, "wood", 60.0, 1, "", 0, 0, 3)

one_more_inning(test_state)

assert_equal(test_state.current_inning, 2)
assert_equal(test_state.misses, 0)
```

My recommendation

For your assignment, I would keep your tests separate from your actual routes. You can put them at the bottom of your file, or in a separate test file if your instructor allows it.

One thing to be careful about: Routes such as actual_batting use randomness, and routes such as index mainly display pages. For those, checking the page type or the state changes is more useful than checking a specific page's exact contents.

You can test your existing routes without adding any new functionality to your baseball game.

If route tests are tricky, code review tools can suggest fixes too.

can i get the complete code to paste a the bottom of my file that tests every function

Worked for 7s

Absolutely. You can paste the following test section into your file. It tests all 12 routes using your existing functions and assert_equal—you don't need to create any new game functionality.

Important: Put this section immediately before your start_server(...) line, not after it. Otherwise, the server may start before the tests run.

Test section

```
test_page = index(test_state)
```

```
# TEST 8: actual_batting
test_state = State([], 0, 50, 500, "wood", 60.0, 1, "", 0, 0, 0)
test_page = actual_batting(test_state)
assert_equal(type(test_page), Page)
assert_equal(test_state.target_row >= 0, True)
assert_equal(test_state.target_row <= 7, True)
assert_equal(test_state.target_col >= 0, True)
assert_equal(test_state.target_col <= 7, True)


# TEST 9: final_score - HIT
test_state = State([], 0, 50, 500, "wood", 60.0, 1, "", 0, 0, 0)
final_score(test_state, 1, 1)
assert_equal(len(test_state.batting_player), 1)
assert_equal(test_state.total_score > 0, True)
assert_equal(test_state.misses, 0)

# TEST 9: final_score - MISS
test_state = State([], 0, 50, 500, "wood", 60.0, 1, "", 0, 0, 0)
final_score(test_state, 9, 9)
assert_equal(len(test_state.batting_player), 1)
assert_equal(test_state.total_score, 0)
assert_equal(test_state.misses, 1)


# TEST 10: one_more_inning
test_state = State([], 0, 50, 500, "wood", 60.0, 1, "", 0, 0, 3)
one_more_inning(test_state)
assert_equal(test_state.current_inning, 2)
assert_equal(test_state.misses, 0)


# TEST 11: finish_game
test_state = State([], 15, 50, 500, "wood", 60.0, 1, "", 0, 0, 0)
test_page = finish_game(test_state)
assert_equal(type(test_page), Page)
assert_equal(test_state.total_score, 15)


# TEST 12: restart
test_state = State([], 25, 50, 500, "wood", 60.0, 4, "", 0, 0, 2)
test_state.batting_player.append(
  AtBatters("Random", "1, 1", 60.0, 3, "HIT", 100)
)

restart(test_state)
assert_equal(test_state.total_score, 0)
```

```
assert_equal(test_state.misses, 0)
assert_equal(test_state.current_inning, 1)
assert_equal(len(test_state.batting_player), 0)
assert_equal(test_state.bat_type, "wood")
```

A couple of things to know

1. These tests assume your route parameters match the ones in the code we've been working with. If your parameter types or names have changed, we may need to adjust the calls.
2. actual_batting uses randomness, so the test checks that the target stays inside the valid grid range rather than expecting a specific location.
3. These tests call your routes directly. They don't test whether the website buttons navigate correctly in a browser.

One recommendation: Run the tests before launching your server. If you get an error, send me the traceback and I can help identify which test needs adjusting.

You're using the free version of ChatGPT. Join your team workspace or upgrade for advanced models, better memory, and higher-quality image creation.